

bigspy 运行速度 / 性能测试报告

- 日期：2026-09-21
- 被测对象：bigspy dev 分支源码；example/run_bigspy_jax.py + benchmarks/benchmark_*.py 全套（8 个脚本）
- 运行方式：conda 环境 bigspy（Python 3.11.16）
- 原始日志：out/example_run.log（端到端）、out/benchmarks/*.log（8 个 benchmark 全部原样保存，含墙钟与退出码）

1. 测试环境

项目	值
CPU	AMD Ryzen 5 3550H, 4 核 / 8 线程, 最高 3.7 GHz
内存	14 GiB（测试时可用 ~11 GiB）
磁盘	NVMe SSD（/dev/nvme0n1p3）
系统	Fedora Linux 44, 内核 7.2.5
Python	3.11.16（conda env bigspy, conda-forge）
numpy / scipy	2.4.6 / 1.17.1
astropy / lmfit	8.0.1 / 1.3.4
ultranest	4.5.2（HDF5 后端, h5py）
jax / jaxlib	0.10.2 / 0.10.2（CPU 后端, 无 GPU）

仓库自带的 benchmarks/benchmark_report.md 基线是 **80 核 Xeon Gold 6248 + 754 GB 内存**。本机是 4 核/8 线程移动 CPU，加速比量级受硬件影响很大，**绝对值不可直接对比**；本报告全部为本机实测。

2. 测试方法

1. **端到端**：/usr/bin/time -v 包裹 python example/run_bigspy_jax.py，记录墙钟、CPU 时间、峰值内存；脚本自身对 SpecFit 与主 MCMC 段计时，分段耗时另有 UltraNest 日志时间戳交叉核对。
2. **微基准**：从仓库根依次运行全部 8 个 benchmarks/benchmark_*.py，每个脚本独占运行（串行，避免争抢 CPU），记录每个脚本的墙钟与退出码。脚本内部自带 old/new parity 断言（超差即非零退出）。
3. **数据**：合成数据来自 tests/_synth.py；真实数据为 LFS 的 MaNGA 测试谱 + BC03 SSP/PCA 模板（本机已具备）。

3. 端到端全流程（example/run_bigspy_jax.py）

3.1 总览

指标	实测值
总墙钟时间	9 分 07 秒 (547.2 s)
用户 CPU 时间	2173.1 s
系统 CPU 时间	35.7 s
平均 CPU 利用率	403% (约 4 个线程)
峰值常驻内存 (Max RSS)	3.16 GiB (3,314,412 KB)
Swap 使用	0

3.2 分阶段耗时（时间戳交叉核对）

阶段	起止时刻	墙钟	似然评估次数	吞吐
加载数据 + SpecFit (mode2)	09:46:20	0.1 s	—	—
主 MCMC — DelayedExp, n_Live=400	09:46:26 → 09:47:53	87 s	23,346	~268 /s
自定义 SFH — DelayTau, n_Live=100→200	09:47:54 → 09:48:06	12 s	5,018	~418 /s
DPL SFH, n_Live=200→400	09:48:07 → 09:55:24	437 s (7.3 min)	593,066	~1,357 /s
绘图 / 保存	—	~11 s	—	—

关键发现：DPL 段占总墙钟约 80%。 它只有 4 个参数，但 UltraNest 自动把 n_Live 从 200 加宽到 400，实际调用似然 **59.3 万次**（主运行仅 2.3 万次）。跳过该段可把全流程压到约 **1.5–2 分钟**。

3.3 实际科学输出

SpecFit (mode2) :

```
v_e = 146.7 ± 5.2 km/s   v_d = 67.7 ± 7.6 km/s   E(B-V) = 0.1880 ± 0.0155
p1 = 0.5929,  p2 = -0.0650
```

主 MCMC (DelayedExp, log Z = -1638.61, posterior N=4813, ESS=945) :

```
t0    = 1.1815  [+1.2983 / -0.8152]  Gyr
tau    = 3.5906  [+0.3079 / -0.4968]  Gyr
logZsun = -0.9162 [+0.0381 / -0.0422]
Z      = 0.00235 (Z_solar = 0.02)
```

模型对比：

模型	参数数	log Z	ΔlogZ
DelayedExp	3	-1638.61	+0.0
DelayTau	2	-1636.17	+2.4
DPL	4	-1636.51	+2.1

略偏好 DelayTau/DPL，但 $\Delta \log Z \approx +2$ ，证据强度很弱。

4. Benchmark 套件总览（全部 8 个，串行运行）

脚本	优化项 / 主题	墙钟	本机加速比	parity
benchmark_sfh	P4 evaluate_batch 向量化	3 s	3.6–6.2×	0（位级）
benchmark_csp	P3 CSP build_batch 分组 dgemm	39 s	合成 1.7–2.4× / 真实 1.6–16.3×	~1e-15
benchmark_conv_matrix	P1 卷积矩阵构建 + 缓存	9 s	构建 2.6–21× / 批量 1.4–3.7×	0（位级）
benchmark_likelihood_interp	P2 插值/掩码预计算	3 s	插值 1.2–2.8× / call_batch 1.5–2.1×	~1e-15
benchmark_mean_filter	P5 滑动窗口向量化	2 s	5.0–49.0×	0（位级）
benchmark_specfit_hotpath	P6 SpecFit 热路径	16 s	calz 2.4× / mode1 1.46× / mode2 10.1× / 整体 1.74× / 真实谱 3.81×	0（位级）
benchmark_mcmc_pipeline	组合 P1+P2+P3+P4（真实谱）	216 s	1.8–2.6×	~1e-15
benchmark_jax	JAX JIT vs NumPy	15 s	2.3–3.3×	~5e-4 (JAX fp32)
合计		~303 s (5 min)	8/8 脚本 退出码 0	

5. Benchmark 明细

5.1 P4 — SFH evaluate_batch（benchmark_sfh）

N	old (s)	new (s)	speedup	max dSFR
10	0.0002	0.0001	3.6x	0.0e+00
100	0.0023	0.0004	6.2x	0.0e+00
1000	0.0243	0.0043	5.6x	0.0e+00
10000	0.2351	0.0502	4.7x	0.0e+00

5.2 P3 — CSP `build_batch` (`benchmark_csp`)

```
== Synthetic SSP (3 metal x 6 age x 761 px) ==
  N | old (s) | new (s) | speedup | max|dCSP| | rel
  10 | 0.0005 | 0.0003 | 1.7x | 1.8e-15 | 4.1e-16
  400 | 0.0160 | 0.0080 | 2.0x | 7.1e-15 | 5.4e-16
  1000 | 0.0582 | 0.0304 | 1.9x | 7.1e-15 | 5.7e-16

== Real SSP (6 x 196 x 3129 px) ==
  N | old (s) | new (s) | speedup | max|dCSP| | rel
  10 | 0.1153 | 0.0733 | 1.6x | 1.0e-17 | 1.3e-15
  50 | 0.4956 | 0.1112 | 4.5x | 1.4e-17 | 2.1e-15
  200 | 2.1088 | 0.1701 | 12.4x | 2.1e-17 | 2.3e-15
  400 | 4.0064 | 0.2453 | 16.3x | 2.1e-17 | 2.8e-15
```

5.3 P1 — 卷积矩阵 (`benchmark_conv_matrix`)

```
== A) 构建: old 双循环 vs 向量化 ==
n_pix | sigma | old (s) | new (s) | speedup | max|dK|
  150 | 1.74 | 0.0011 | 0.0001 | 8.0x | 0.0e+00
  150 | 8.00 | 0.0033 | 0.0002 | 16.6x | 0.0e+00
  3129 | 1.74 | 0.0382 | 0.0145 | 2.6x | 0.0e+00
  3129 | 8.00 | 0.0970 | 0.0197 | 4.9x | 0.0e+00

== B) gauss_convolve_batch (N, 3129): 每次重建 vs 缓存 ==
  N | old (s) | new 1st (s) | new cached (s) | speedup | max|dout|
  10 | 0.0696 | 0.0322 | 0.0189 | 3.7x | 0.0e+00
  50 | 0.0804 | 0.0393 | 0.0235 | 3.4x | 0.0e+00
  200 | 0.1199 | 0.0826 | 0.0737 | 1.6x | 0.0e+00
  400 | 0.2126 | 0.1825 | 0.1565 | 1.4x | 0.0e+00
```

本机与 80 核基线分歧最大的一项：基线报告稳态批量卷积为 9–92×，本机只有 1.4–3.7×。原因是这里 `mtx` 大矩阵乘法才是主导开销（3129×3129 矩阵），而旧代码“每次重建矩阵”的相对惩罚在这颗 CPU 上小得多。这正是硬件相关的典型表现。

5.4 P2 — 插值预计算 (`benchmark_likelihood_interp`)

```
== A) 批量插值: interp1d (每次重建) vs plan.apply ==
  N | old (s) | new (s) | speedup | max|d|
  10 | 0.0006 | 0.0002 | 2.8x | 0.0e+00
  1000 | 0.0392 | 0.0314 | 1.2x | 0.0e+00

== B) Likelihood.call_batch, vd=0, 偏移观测网格 (合成 SSP) ==
  N | old (s) | new (s) | speedup | max|dchi2| | rel
  10 | 0.0020 | 0.0013 | 1.5x | 1.2e-10 | 5.5e-16
  1000 | 0.0851 | 0.0411 | 2.1x | 1.5e-10 | 6.9e-16
```

5.5 P5 — `mean_filter` (`benchmark_mean_filter`)

case	win	old (s)	new (s)	speedup	max d
unmasked	200	0.0168	0.0003	49.0x	0.0e+00
mask=all-1	200	0.0212	0.0027	7.9x	0.0e+00
mask=random	200	0.0214	0.0022	9.7x	0.0e+00
unmasked	400	0.0156	0.0004	37.9x	0.0e+00
mask=all-1	400	0.0220	0.0044	5.0x	0.0e+00
mask=random	400	0.0208	0.0039	5.3x	0.0e+00

5.6 P6 — SpecFit 热路径 (benchmark_specfit_hotpath)

```
== A) 每次 lmfit 迭代的尘埃曲线 (合成 ~3400 px) ==  
old calz_unred : 142.9 us/call  
new 10**(0.4*klam*(-ebv)) : 60.2 us/call -> 2.4x, max|d| = 0.0e+00  
== B) run_mode1 端到端 : old 1.141 s | new 0.783 s | 1.46x  
== C) run_mode2 端到端 : old 0.266 s | new 0.026 s | 10.12x  
== D) fit_spectrum(mode='both') : old 1.408 s | new 0.810 s | 1.74x  
== E) 真实 MaNGA 谱 SpecFit.fit(mode2) : old 0.32 s | new 0.08 s | 3.81x
```

5.7 组合热路径 (benchmark_mcmc_pipeline, 真实 MaNGA + 真实 SSP)

```
vd=67.7 km/s (sigma_pix=0.98), n_wave_ssp=13160, n_obs=3129  
N | old (s) | new (s) | speedup | max|dchi2| | rel  
10 | 0.7691 | 0.4236 | 1.8x | 3.6e-11 | 9.5e-16  
50 | 1.3741 | 0.6081 | 2.3x | 1.5e-10 | 1.5e-15  
200 | 3.9274 | 1.5180 | 2.6x | 1.2e-10 | 3.3e-15  
400 | 7.9392 | 3.2648 | 2.4x | 2.6e-10 | 3.0e-15  
1000 | 19.3335 | 7.9357 | 2.4x | 1.7e-10 | 6.6e-15
```

5.8 JAX vs NumPy (benchmark_jax)

N	NumPy (s)	JAX 1st (s)	JAX mean (s)	Speedup	Δrel
10	0.0524	0.5817	0.0165	3.2x	1.59e-04
50	0.0752	0.5395	0.0230	3.3x	4.89e-04
200	0.1442	0.6257	0.0567	2.5x	3.78e-04
500	0.3580	0.6173	0.1398	2.6x	4.67e-04
1000	0.6498	0.7614	0.2857	2.3x	4.79e-04

6. 生成的实际产物

图片 (out/figs/, 均为有效 PNG, 已逐一校验尺寸)

文件	尺寸	内容
01_observed_spectrum.png	1227×381	观测谱
02_specfit.png	1428×707	SpecFit 拟合结果
02b_dust_curve.png	1096×606	尘埃曲线
03_corner.png	879×892	主 MCMC 后验 corner
04_bestfit_csp.png	1213×484	最佳拟合 CSP 谱
05_sfh_ci.png	838×476	SFH 68% 置信区间
06_corner_custom_sfh.png	636×640	DelayTau corner
06b_sfh_delaytau.png	838×452	DelayTau SFH CI
07_corner_dpl.png	1131×1145	DPL corner (beta 约束弱, 有 contour 警告)
07b_sfh_dpl.png	838×452	DPL SFH CI

数据 / 链 / 日志

- out/data/: specfit_result.fits、mcmc_bestfit.fits、posterior_samples.npy

- `out/chains_manga-7443/` (3.1 MB)、`out/chains_custom_sf/` (808 KB)、`out/chains_dpl/` (3.5 MB)
 - `out/example_run.log` (端到端原始日志)
 - `out/benchmarks/*.log` (8 个 benchmark 原始日志 + `ALL.log` 汇总)
-

7. 发现与建议

性能

1. **DPL 段是端到端唯一瓶颈**：占 80% 墙钟、59 万次似然调用。不需要模型对比时删掉/单独跑第 6b 节，全流程可降到约 1.5–2 分钟；或给 DPL 设更小 `n_live` 并禁止加宽。
2. **CPU 仅用 ~4 线程 (403%)**：嵌套采样本身较串行，JAX/NumPy 未吃满 8 线程；调 `OMP_NUM_THREADS / XLA_FLAGS` 收益有限。
3. **P1 缓存收益高度依赖硬件**：本机 `gauss_convolve_batch` 仅 1.4–3.7× (基线 9–92×)，因为这里 `K@X` 大矩阵乘占主导。若目标机型核少/BLAS 弱，P1 的"重建矩阵"惩罚相对不突出。
4. 其余优化项 (P3 真实谱 16.3×、P5 49×、P6 mode2 10.1×、JAX 2.3–3.3×) 在本机均能复现，量级合理。

数值

1. **JAX 与 NumPy 的 chi2 相对差约 5e-4** (JAX 默认 float32) ——MCMC 采样走 float32，最终绘图/保存走 NumPy float64；对高精度后验敏感时可开 `jax_enable_x64`。
2. 本机 `benchmark_csp / benchmark_likelihood_interp` 的 parity 不是精确 0 (~1e-15)，而基线 Xeon 上是 0 ——这是不同 BLAS 下 `dgemm/dgemv` 求和顺序差异所致，**均在脚本断言容差内 (8/8 退出码 0)**。

可维护性 / 数据

1. **【已修复】三次 UltraNest 运行共用同一个 `debug.log`**：原因是 UltraNest 的 `create_logger` 只在进程内首次调用时挂载 `debug.log` `FileHandler` (依据 `logger.handlers` 判断)，因此后两次运行仍写进 `out/chains_manga-7443/debug.log`，另外两个 chain 目录没有各自日志。修复：在 `UltraNestSampler.run()` 开头调用新增的 `_reset_ultranest_log_handlers()`，解绑并关闭旧 handler，让当前 `log_dir` 各自生成 `debug.log`；并新增回归测试 `tests/test_sampler.py::TestRun::test_each_run_writes_its_own_debug_log`。
 2. **【已修复】`out/figs/dust_curve_manga-7443.png` 实为 Git-LFS 指针文本** (并非 PNG, 130 B)，且全仓库无任何代码引用，属遗留孤儿文件；已删除 (脚本实际输出为 `02b_dust_curve.png`)。
-

8. 复现命令

```
source ~/miniconda3/etc/profile.d/conda.sh
conda activate bigspy
cd /home/chengz/Code/bigspy

# 端到端全流程 (§3)
MPLBACKEND=Agg /usr/bin/time -v python example/run_bigspy_jax.py

# 全部 benchmark (§4-§5, 串行, 约 5 分钟)
for b in benchmark_sfh benchmark_csp benchmark_conv_matrix \
    benchmark_likelihood_interp benchmark_mean_filter \
    benchmark_specfit_hotpath benchmark_mcmc_pipeline benchmark_jax; do
    echo "== $b =="; python "benchmarks/$b.py"
done
```